

IMPACT ANALYSIS

HIGH RISK

COMPLEXITY: HIGH

NovaPeak Financial Services is consolidating 4 legacy systems into a unified Salesforce Customer 360 platform serving 380,000 customers and 450 agents across Australia/NZ. The solution integrates Service Cloud, Experience Cloud, Apex automation, and real-time APIs to legacy banking, policy, and data warehouse systems with significant data volumes (3M cases, 10M integration logs projected over 5 years).



Solution Architect

SKIPPED

Review mode — design already exists



Apex Engineer

REQUIRED

WHY TRIGGERED

Critical trigger architecture, synchronous CBS callout pattern, batch job design, governor limit risk at scale, error handling gaps.

WILL CHECK

Synchronous callout from Case trigger at 120 cases/hr peak volume

FinancialProductTrigger CPU time untested at 60K daily records

Est. ~40s

~\$0.020



Integration Architect

REQUIRED

WHY TRIGGERED

Three bidirectional integrations (CBS, PMS, CDW), Outbound Messaging + REST API patterns, payload encryption, retry logic gaps, Platform Events design.

WILL CHECK

CBS inbound 800 msgs/min peak load via synchronous Apex handler

PMSSyncBatch rollback entire 200-record chunk on single failure

Est. ~40s

~\$0.020



Flow Automation Specialist

REQUIRED

WHY TRIGGERED

Platform Events triggering internal flows, SLA calculation flow, Account update and escalation notification flows, automation patterns.

WILL CHECK

Escalation and account update flows as subscribers to Platform Events

SLA monitoring every 15 minutes at 5K case volume

Est. ~35s

~\$0.020



LWC Specialist

REQUIRED

WHY TRIGGERED

Experience Cloud portal with 5 custom LWC components, 2,400 concurrent sessions, polling pattern, guest user API access, authentication design.

WILL CHECK



Architecture Patterns Expert

REQUIRED

WHY TRIGGERED

One-trigger-per-object pattern, event-driven architecture assessment, LDV strategy (3M cases, 10M logs), scalability at financial services scale, Well-Architected alignment.

WILL CHECK

caseTrackerTimeline 30-sec polling at 2,400 concurrent users
= API load spike

notificationPreferences guest user Write access to Contact object

Est. ~28s

~\$0.020

Integration_Log__c 10M record projection with no archiving strategy

Case volume 3M over 5 years requires LDV/archiving roadmap

Est. ~35s

~\$0.020

**OmniStudio Specialist**

OPTIONAL

WHY TRIGGERED

Solution uses Flow automation; OmniStudio assessment for multi-step case submission and omni-channel context management recommended.

WILL CHECK

Case submission form uses custom LWC; OmniScript alternative not evaluated

Est. ~30s

~\$0.020

**Architecture Review Board**

REQUIRED

WHY TRIGGERED

Final ARB verdict on high-risk items: synchronous callouts, guest user security, polling load, governor limits, compliance with APRA CPS 234.

WILL CHECK

6 open items including 2 high-severity (CBS callout, guest user access)

APRA compliance sign-off required before go-live

Est. ~45s

~\$0.020

**ADR Documentation Lead**

REQUIRED

WHY TRIGGERED

Document architectural decisions on integration patterns, security model, data residency, trigger design, and risk resolutions.

WILL CHECK

Design decisions on CBS synchronous vs async patterns not documented

Trade-offs on guest user access vs security model not captured

Est. ~20s

~\$0.020

**Session Learning Facilitator**

REQUIRED

WHY TRIGGERED

Extract session learnings from ARB review on financial services integrations, large data volumes, security patterns, and Salesforce scalability.

WILL CHECK

Complex integration patterns warrant team knowledge capture

Governor limit strategy at scale valuable for future engagements

Est. ~18s

~\$0.020

CROSS-CUTTING CONSIDERATIONS

- › Synchronous CBS callout in trigger context is critical risk — recommend immediate async pattern redesign (Queueable or Platform Event-driven handler) pre-go-live.
- › PMSSyncBatch error partitioning gap creates silent data loss risk — implement per-record error handling and Integration_Log__c flagging before deployment.
- › Guest User Create/Edit on Contact violates zero-trust security principle — use tokenised API with field-level security instead of broad profile permissions.
- › caseTrackerTimeline polling at 2,400 concurrent users will exceed API limits — redesign as server-sent events (Platform Events subscription) or reduce polling interval to 60 seconds.
- › Financial_Product__c risk rating CPU time (800ms per 200 recs) untested at 60K daily load — conduct volume testing immediately; consider async batch if CPU exceeds limits.
- › Integration_Log__c projected 10M records in 5 years with no archiving — define retention policy and automated purge strategy (weekly cleanup job is insufficient).

- APRA CPS 234 compliance requires formal data encryption audit — validate Shield Platform Encryption key rotation, field coverage, and audit logging before go-live.
- Australian Privacy Act 1988 obligations (right to access/delete PII) not addressed in design — define GDPR-equivalent data subject request workflow.
- One-trigger-per-object pattern with 4 active triggers on high-volume objects — monitor for trigger execution order dependencies and recursion risks.
- Named Credential for CBS authentication requires rotation policy — document credential lifecycle management and access controls.
- Experience Cloud guest user session timeout and MFA enforcement not specified — confirm APRA alignment with 24/7 portal access model.

AGENT ROSTER



Designer
SOLUTION ARCH
skipped



LWC
UI SPECIALIST
✓ 14s



OmniStudio
OMNI EXPERT
skipped



Flow
FLOW BUILDER
✓ 16s



Apex
APEX EXPERT
✓ 13s



Patterns
PATTERNS
✓ 15s



Integration
INTEGRATION
✓ 15s



Judge
JUDGE
✓ 15s



Scribe
SCRIBE
✓ 12s



Learner
LEARNER
✓ 15s

SPECIALIST REVIEWS



LWC & UI Specialist

UI SPECIALIST

14s

LWC & Experience Cloud – Specialist Domain Review

NovaPeak Financial Services | Customer 360 Platform v1.2

Reviewer Domain: Lightning Web Components, Experience Cloud, Portal UI Architecture

Review Date: June 2026

COMPONENT DESIGN

Assessment of Proposed Component Hierarchy

The design proposes five custom LWC components deployed to an LWR Experience Cloud site. The component hierarchy and data flow are **partially specified but contain critical gaps** that prevent ARB sign-off from a UI architecture perspective.

Proposed components assessed:

Component	Design Adequacy	Risk Level
accountSummaryWidget	Underspecified — CBS callout pattern on load is high-risk	Critical
caseSubmissionForm	Acceptable pattern described; detail insufficient	Medium
caseTrackerTimeline	Polling pattern is architecturally unacceptable at stated scale	Critical
documentUploadWidget	Content delivery enabled on financial documents is a security concern	Critical
notificationPreferences	Guest User write pattern is a critical security defect	Critical

Missing from the component design entirely:

- > No component target declarations specified (`lightningCommunity__Page` , `lightningCommunity__Default`)
 - > No LMS (Lightning Message Service) or inter-component communication strategy defined
 - > No error boundary pattern described — what does a customer see when CBS is unavailable?
 - > No loading state / skeleton screen strategy for components with async data dependencies
 - > No component-level caching strategy
-

IMPLEMENTATION APPROACH

MUST-FIX #1 — `caseTrackerTimeline` : 30-Second Polling is Architecturally Unacceptable

What the design says:

> "Polls for updates every 30 seconds via `@wire`"

Why this is wrong on multiple levels:

First, `@wire` does not poll. This statement reveals a fundamental misunderstanding of the LWC wire service. `@wire` is a reactive data binding mechanism — it does not re-execute on a timer. The team is almost certainly describing a `setInterval` calling an imperative Apex method, which is an anti-pattern explicitly called out in the Well-Architected guardrails for this engagement.

Second, the scale arithmetic makes this unacceptable regardless of implementation:

2,400 concurrent portal sessions

× 1 Apex callout per 30 seconds
= 4,800

Flow & Automation Domain Review

NOVAPEAK FINANCIAL SERVICES – CUSTOMER 360 PLATFORM (V1.2)

FLOW DESIGN

The design proposes a **mixed automation model**: Apex Triggers with handler classes for core object logic, and Flows for Platform Event subscribers (Account update Flow, Escalation notification Flow, Dashboard refresh trigger). The declarative-first principle is stated in Section 2.2 and partially honoured, but the automation layer has structural gaps that require resolution before go-live.

The Flow components explicitly named in the design are:

- > [CBS_AccountUpdate__e](#) subscriber Flow (Account updates)
- > [Case_Escalation__e](#) subscriber Flow (Escalation notifications)
- > [PMS_SyncComplete__e](#) subscriber Flow (Dashboard refresh)

No Screen Flow or Record-Triggered Flow is described in the automation section, despite the design referencing case routing and self-service form submission — both of which would typically involve Flow in a declarative-first architecture.

TRIGGER & ENTRY CRITERIA

What is designed

Four Apex triggers with handler classes (Case, Account, Financial_Product__c, Contact). One-trigger-per-object pattern is adopted — this is correct and aligns with decision D3 from ARBOARD-17.

What is missing

The design does not define **entry criteria or guard conditions** for any of the Platform Event-triggered Flows. For [CBS_AccountUpdate__e](#) in particular — receiving 15,000–45,000 events per day with peaks of 800/minute — the absence of documented entry criteria, resume/replay strategy, and flow fault handling is a critical gap.

There is also **no documented Record-Triggered Flow** on any object, yet the design claims declarative-first automation (D6, ARBOARD-17). If case routing is handled entirely in [CaseTriggerHandler](#) , that contradicts the declarative-first principle without justification.

LOGIC WALKTHROUGH

Platform Event Flows – Structural Concerns

`CBS\_AccountUpdate\_\_e` → Account update Flow

The design states CBSInboundHandler publishes this event after processing inbound CBS messages. The subscriber Flow then updates Account records. This creates a sequencing risk:

- 1. CBSInboundHandler upserts Account/Financial_Product__c records synchronously
- 2. It then publishes `CBS_AccountUpdate__e`
- 3. The subscriber Flow fires asynchronously and also updates Account

If both the Apex handler and the subscriber Flow write to Account, there is a **dual-write pattern on the Account object** — Apex trigger logic and Flow logic both modifying the same record in overl



Apex & Integration Engineer

APEX EXPERT

13s

Apex & Automation Domain Review

NOVAPEAK FINANCIAL SERVICES – CUSTOMER 360 PLATFORM V1.2

Reviewer Role: Senior Apex & Automation Engineer

Review Date: June 2026

Verdict: ✗ NOT APPROVED — MUST-FIX ISSUES IDENTIFIED

CODE DESIGN

Trigger Architecture Assessment

The one-trigger-per-object pattern with handler class dispatch (Section 5.1) is **accepted** and consistent with ARB decision D3 from ARBOARD-17. The four trigger/handler pairs are structurally sound. However, the handler implementations contain critical defects that must be resolved before go-live.

The following class structure is required as a minimum viable pattern for the CBS callout path, which is the highest-risk component in the design:

```
/**
 * CaseTrigger – one trigger per object, no logic in trigger body
 * Dispatches to CaseTriggerHandler via TriggerDispatcher
 */
```

```
trigger CaseTrigger on Case (before insert, before update, after insert, after update) {  
    TriggerDispatcher.run(new CaseTriggerHandler());  
}
```

```
```apex
```

```
/**
```

```
 * CaseTriggerHandler
```

```
 * Handles SLA calculation, CBS callout on resolution, Platform Event publish on escalation.
```

```
 *
```

```
 * CRITICAL: CBS callout MUST NOT be synchronous from trigger context.
```

```
 * This implementation replaces the synchronous callout pattern described in Section 4.2.2
```

```
 * with a Queueable chain to satisfy governor limits and bulkification requirements.
```

```
 */
```

```
public with sharing class CaseTriggerHandler implements ITriggerHandler {
```

```
 // CBS-actionable resolution categories — must be driven by Custom Metadata,
```

```
 // not hardcoded strings. See CBSActionCategory__mdt.
```

```
 private static final String CALLOUT_TRIGGER_FIELD = 'Resolution_Category__c';
```

```
 public void beforeInsert(List<SObject> newRecords) {}
```

```
 public void beforeUpdate(List<SObject> newRecords, Map<Id, SObject> oldMap) {}
```

```
 public void afterInsert(List<SObject> newRecords) {}
```

```
 public void afterUpdate(List<SObject> newRecords, Map<Id, SObject> oldMap) {
```

```
 List<Case> updatedCases = (List<Case>) newRecords;
```

```
 Map<Id, Case> oldCases = (Map<Id, Case>) oldMap;
```

```
 List<Id> cbsActionableC
```



## Architecture Patterns Advisor

PATTERNS

15s

### Architecture Patterns Review — NovaPeak Financial Services Customer 360 Platform

**Reviewer Role:** Salesforce Enterprise Architecture Patterns Expert

**Document Under Review:** Solution Design Document v1.2 — NovaPeak Financial Services

**Review Date:** June 2026

**ARB Session:** ARBOARD-17

---

#### PATTERN RECOMMENDATION

The following named patterns are assessed against what the design has proposed. Where the design applies a pattern correctly, this is confirmed. Where the design deviates from or omits a required pattern, a remediation is specified.

#### Patterns Assessed:

1. **Bulkified Trigger Handler Pattern** (One-trigger-per-object + Handler Dispatch) — \*Partially Applied\*
2. **Async-First Integration Pattern** — \*Violated (CBS outbound callout)\*
3. **Event-Driven Fault Isolation Pattern** — \*Partially Applied\*
4. **Attribute-Based Access Control (ABAC) Pattern** — \*Violated (Guest User write)\*
5. **Large Data Volume (LDV) Archival Pattern** — \*Absent\*
6. **Idempotent Upsert Pattern** — \*Partially Applied\*
7. **Error Observability Pattern** — \*Partially Applied\*
8. **Polling Anti-Pattern** — \*Present and unmitigated\*

---

#### WHY THIS PATTERN ASSESSMENT

The design demonstrates architectural awareness in several areas — the one-trigger-per-object handler pattern, Named Credentials, Platform Events for internal pub/sub, and LWR template selection are all sound decisions consistent with Salesforce Well-Architected Framework principles. The design team has also correctly self-identified most of the critical risks in Section 8.

However, self-identification without remediation is architecturally insufficient. Five of the eight open items represent pattern violations that will cause production failures at NovaPeak's stated volumes. The analysis below treats each as a first-class architectural defect, not a delivery risk to be deferred.

---

#### IMPLEMENTATION BLUEPRINT

##### CRITICAL ISSUE 1 – Synchronous Callout from Apex Trigger (Open Item #1)

**Pattern Violated:** Async-First Integration Pattern

##### The Problem:

The design describes `CaseTriggerHandler` making a synchronous REST callout to CBS when a Case is resolved with a CBS-actionable `Resolution_Category__c`. This is a compound violation:

- › Apex triggers with synchronous callouts cannot be invoked from batch contexts, meaning `PMSSyncBatch` or any future bulk Case update will fail silently or throw a `System.CalloutException`
- › At the stated



Integration Architecture Review

NOVAPEAK FINANCIAL SERVICES – CUSTOMER 360 PLATFORM V1.2

ARB Submission: ARBOARD-17 | Reviewer: Principal Integration Architect

---

INTEGRATION ASSESSMENT

This design integrates a Salesforce Customer 360 platform (Service Cloud + Experience Cloud) with three downstream systems — CBS (Core Banking), PMS (Policy Management), and CDW (Customer Data Warehouse) — serving 380,000 customers and 450 agents under APRA CPS 234 and Australian Privacy Act obligations. Integration flows are bidirectional at significant volume (up to 800 CBS messages/minute at peak), making integration architecture a primary risk vector for this engagement. The design as submitted contains five integration-domain defects that individually would warrant rejection; collectively they represent an unacceptable risk profile for a regulated financial services go-live.

---

RECOMMENDED INTEGRATION PATTERN

The current design relies on **synchronous Apex trigger callouts and legacy Outbound Messaging** where the correct patterns are:

Flow	Recommended Pattern	Rationale
CBS → Salesforce (inbound)	<b>Platform Events + Apex Subscriber</b>	Decouples CBS push from Salesforce processing; replay, dead-letter, async fan-out
Salesforce → CBS (outbound on case resolution)	<b>Queueable Apex callout chain</b> with retry	Removes callout from trigger context; supports retry, correlation ID, error isolation
PMS → Salesforce (nightly sync)	<b>Bulk API 2.0 via middleware + error partitioning</b>	Eliminates chunk-level rollback; supports 60k+ records safely
CDW → Salesforce (initial load)	<b>Bulk API 2.0</b>	Volume (950k Financial_Product__c) requires Bulk API, not REST
Internal pub/sub	<b>Platform Events (retain as designed)</b>	Correct pattern — accepted

---

ARCHITECTURE DESIGN

Current State vs. Required State

CURRENT (as designed — REJECT):

CBS → SOAP Outbound Message → CBSInboundHandler (sync Apex) → Account/Financial\_Product\_\_c DML  
Case Update → CaseTrigger (sync) → CBS REST callout (no retry, no async)  
PMS → PMSSyncBatch (chunk rollback, no error partition)

REQUIRED STATE:

CBS → REST/SOAP push → Middleware (MuleSoft/equivalent) → Platform Event CBS\_AccountUpdate\_\_e  
→ Apex Subscriber → upsert (async, idemp)

JUDGE RULING

 **Architecture Review Judge**

JUDGE

15s

ARB DRAFT RECOMMENDATION

Recommendation: REJECT

SUMMARY OF FINDINGS

The NovaPeak Financial Services Customer 360 Platform v1.2 design demonstrates sound structural intent — single-org strategy, one-trigger-per-object pattern, Named Credentials, LWR template, and Platform Events for internal pub/sub are all accepted decisions consistent with prior ARB rulings. However, all four specialist agents independently identified the same five critical defects, and the design team's own Section 8 self-identifies these risks without providing remediation paths. Self-identification without remediation is architecturally insufficient for a CPS 234-regulated go-live serving 380,000 customers and 450 agents. The design cannot proceed to build in its current state.

CRITICAL ISSUES (MUST FIX)

MUST FIX:

1. **Synchronous CBS callout from `CaseTrigger` context must be replaced with Queueable Apex.**

The design acknowledges this risk in Open Item #1 but defers resolution. A synchronous callout from a trigger context will fail silently when invoked from any batch context (including `PMSSyncBatch` or `IntegrationLogCleanup` ), has no retry mechanism, and will cause complete transaction rollback on CBS timeout. This is a compound violation of the Async-First Integration Pattern and directly maps to failure pattern **FP-004** (no governor limit analysis upfront) and **FP-006** (no error logging with resolution path). Required remediation: replace with

Queueable Apex callout chain with correlation ID, retry counter driven by Custom Metadata, and fault logging to [Integration\\_Log\\_\\_c](#) with an explicit resolution runbook. CBS-actionable resolution categories must be driven by [CBSActionCategory\\_\\_mdt](#) — no hardcoded strings.

## 2. Guest User write access to `Contact` must be removed immediately.

The [notificationPreferences](#) component grants the Experience Cloud Guest User profile Create and Edit permissions on the Contact object to support unauthenticated consent updates via tokenised URL. This is a critical security defect in a CPS 234-regulated environment. A misconfigured or replayed token allows any unauthenticated actor to overwrite Contact records for any customer. Required remediation: replace with an authenticated low-friction flow (re-authentication via one-time passcode or verified session token validated server-side in Apex), or implement a dedicated [ConsentUpdateRequest\\_\\_c](#) staging object with restricted Guest User access scoped only to that object, with an Apex-validated token check before any Contact write is permitted. Guest User must never hold Create/Edit on core CRM objects.

3

## SCRIBE & LEARNING

**ARB Scribe**

SCRIBE

12s

### ARCHITECTURE DECISION RECORD

#### ADR-ARBOARD-17-001

**Date:** 2026-06-27**Status:** Proposed — REJECT**Confidence Level:** High

---

#### Context

NovaPeak Financial Services requires a unified Salesforce Customer 360 platform consolidating customer data across four legacy systems (CBS, PMS, CDW, and one unspecified source) for 380,000 customers and 450 internal agents. The solution spans Service Cloud, Experience Cloud (LWR), Apex, Flow, and Platform Events, operating under Australian Privacy Act 1988 and APRA CPS 234 obligations in the ap-southeast-2 (Sydney) data region.

The design is presented at v1.2 draft. The ARB Judge has issued a **REJECT** verdict. Five unmitigated critical defects make the design unfit for go-live in a CPS 234-regulated environment. The design team self-identified the majority of these risks in Section 8 (Open Items) but provided no remediation paths. Self-identification without remediation is insufficient for ARB sign-off.

The following architectural decisions embedded in the current design are assessed as follows:

#	Decision	ARB Status
D1	Single-org strategy (Service Cloud + Experience Cloud)	Accepted
D2	LWR site template	Accepted
D3	One-trigger-per-object with handler class pattern	Accepted
D4	Named Credentials for CBS outbound callout	Accepted
D5	Platform Events for internal pub/sub	Accepted
D6	Declarative-first automation (Flow for case routing)	Accepted
D7	Synchronous CBS callout from <a href="#">CaseTrigger</a>	Rejected
D8	Guest User Create/Edit on <a href="#">Contact</a> for consent updates	Rejected
D9	PMSSyncBatch without error partitioning	Rejected — incomplete
D10	<a href="#">caseTrackerTimeline</a> 30-second polling pattern	Rejected
D11	<a href="#">FinancialProductTrigger</a> CPU risk unmitigated	Rejected — incomplete

---

Decision

The design is **REJECTED**. Implementation must not proceed until all Critical Issues listed below are resolved and a revised design is resubmitted for ARB review.

Critical Issues — Must Fix Before Re-submission

Org Intelligence Learner

LEARNER

15s

I'll work through this systematically: first complete my independent assessment, then extract learnings, then reconcile prior ADRs.

---

NEW LEARNINGS FROM THIS SESSION

- Synchronous callouts from trigger contexts are a compound failure risk in regulated financial environments** — they fail silently in batch contexts, lack retry mechanisms, and cause full transaction rollback on timeout; Queueable Apex with a correlation ID and Custom Metadata-driven retry counter is the required remediation pattern for CPS 234-regulated orgs.
- Guest User profile must never hold Create or Edit permissions on core CRM objects (Account, Contact, Case)** — even when protected by tokenised URLs, unauthenticated write access to Contact in a regulated environment constitutes a critical security defect; consent updates must route through a scoped staging object with server-side Apex token validation, or through a re-authenticated low-friction session.

3. **Self-identified risks in a design document without accompanying remediation paths are architecturally insufficient for ARB sign-off** — a Section 8 open items list that names critical defects but defers resolution does not satisfy ARB gate criteria in CPS 234-regulated engagements; remediation paths must be present in the submitted design.
4. **30-second polling via `@wire` on a portal component is an anti-pattern at scale** — at 2,400 concurrent portal sessions this generates ~4,800 API calls per minute from a single component; the approved remediation pattern for real-time portal updates in this org is Platform Events with a Lightning Message Service subscriber, not interval polling.
5. **Batch jobs processing high-volume custom objects (>500K records/year) require an explicit archiving and storage strategy before go-live** — `Integration_Log__c` projected at 10M records over five years will breach storage limits without a defined archiving runbook; this must be resolved at design time, not post-go-live.
- 

PATTERNS CONFIRMED

- > **Declarative-first automation (Flow over Apex)** — confirmed again via accepted Decision D6; case routing and account update flows remain the approved pattern for status-transition logic at this complexity level. Consistent with ARBOARD-12 through ARBOARD-15.
- > **Single-org strategy** — accepted as Decision D1 for the NovaPeak engagement; consistent with the established default across both the NovaPeak and Telco engagements in this org's history.
- > **Platform Events for internal pub/sub** — accepted as Decision D5; `CBS_AccountUpdate__e` , `Case_Escalation__e` , and `PMS_SyncComplete__e` follow

Agent Dissent Analysis

0 of 6 agents dissented

▲

AGENT	RISK	CONCERN	VERDICT
Salesforce Solution Designer	HIGH	Design contains five critical defects acknowledged in Section 8 but without remediation paths	✓ Aligned
LWC & UI Specialist	HIGH	Four critical UI defects: synchronous CBS callout pattern, 30-second polling anti-pattern, Guest User write to Contact, missing component security boundaries	✓ Aligned
Flow & Automation Specialist	HIGH	Missing entry criteria and resume/replay strategy for 15,000–45,000 Platform Event flows daily; dual-write pattern on Account object from both Apex and Flow	✓ Aligned
Apex & Integration Engineer	HIGH	Synchronous CBS callout from CaseTrigger context will fail silently in batch contexts, has no retry mechanism, violates Async-First Integration Pattern (FP-004, FP-006)	✓ Aligned

Architecture Patterns Advisor

HIGH

Five first-class pattern violations: synchronous callout, Guest User ABAC violation, polling anti-pattern, missing LDV archival, incomplete event-driven fault isolation

✓ Aligned

Integration Architect

HIGH

Synchronous Apex trigger callouts and legacy Outbound Messaging violate Async-First Integration Pattern; CBS callout from trigger context fails silently in batch contexts with no retry

✓ Aligned

All four specialist agents independently identified the same five critical defects (synchronous CBS callout, Guest User write access, polling anti-pattern, missing entry criteria, LDV archival), completely aligning with the Judge's REJECT verdict and must-fix remediation requirements.

ENDORSEMENT DECISION

✓ Jira ticket created  
[ARBOARD-18](#)

✓ ADR saved to Jira – [ARBOARD-18](#)

SESSION ACTUALS VS ESTIMATE

TOTAL TIME  
2m 6s

TOKENS USED  
37.4k  
est. 41k

ACTUAL COST  
\$0.304  
est. \$0.196